

RTL Design Sherpa

Bridge Generator Architecture Specification 1.1

October 15, 2026

Table of Contents

1 Bridge Gas Index....11
2 Document Information....12
2.1 Bridge Generator Architecture Specification....12
2.2 Revision History....13
2.3 Document Purpose....15
2.4 Intended Audience....16
2.5 Related Documents....17
3 What the Generator Produces....18
3.1 Generated Artifact Family....19
3.2 What Each Variant Means....20
3.3 Scope of Generation....21
4 When to Regenerate....22
4.1 Triggers for Full Regeneration....23
4.2 Regeneration Workflow....24
4.3 When Inputs (Not Code) Change....25
4.4 Identifying Generated Files....26
5 Generator Entry Points....27
5.1 Single-Bridge Mode....28
5.2 Bulk Mode....29
5.3 Makefile Targets....30
5.4 Return Value....31
6 Bridge-Level TOML Keys....32

6.1 [bridge] scalar keys....33
6.2 [bridge.clock] table....34
6.3 Crossbar and arbitration keys....35
6.4 Monitor and cfg subsystem keys....37
6.5 [bridge.mon_group] table....39
6.6 [bridge.defaults] table....40
7 Port Specification Schema....41
7.1 Common port fields....42
7.2 Slave-only fields....44
7.3 Optional behavior fields....45
7.4 axi5_features values....47
7.5 Channel semantics....48
7.6 Example....49
8 Interface Wrapper Configuration....50
8.1 interface.type values....51
8.2 interface.skid_depths table....52
8.3 latency (port key)....53
8.4 How the wrapper selection flows....54
8.5 Protocol-specific wrapper notes....55
9 Monitor and CFG Subsystem Keys....56
9.1 Monitor subsystem....57
9.1.1 Bridge-level monitor keys....57
9.1.2 Per-port monitor keys....57
9.1.3 [bridge.mon_group] table....58

9.2 CFG register block..... 59
9.2.1 What changes..... 59
9.2.2 Per-port cfg fields..... 59
10 Validation Rules..... 61
10.1 Field validation (validate_required_fields)..... 62
10.2 Channel and protocol validation..... 63
10.3 Protocol-specific constraints..... 64
10.3.1 APB / APB5 (validate_apb_constraints)..... 64
10.3.2 Wishbone B4 (validate_wb4_constraints)..... 64
10.3.3 AXI3 (validate_axi3)..... 64
10.3.4 AXI5 (validate_axi5)..... 64
10.4 Master-slave compatibility (validate_master_slave_compatibility)..... 65
10.5 Address map validation (validate_address_map)..... 66
10.6 Slave ID width (validate_slave_id_widths)..... 67
10.7 CDC constraints (validate_cdc_constraints)..... 68
10.8 Warning for dropped sideband (warn_axi5_dropped_sideband)..... 69
11 Pipeline Overview..... 70
11.1 Pipeline diagram..... 71
11.1.1 Figure 3.1: Bridge Generator Pipeline..... 71
11.2 Key files by stage..... 72
12 Configuration Loading..... 74
12.1 TOML parsing..... 75
12.2 Connectivity CSV..... 76
12.3 BridgeConfig construction..... 77

12.4 Normalization....78
13 Validation....79
13.1 Validation order....80
13.2 Error behavior....81
13.3 Examples of validation failures....82
13.4 Adding a new validation rule....83
14 Adapter and Crossbar Emission....84
14.1 Translation to emitter structs....85
14.2 BridgeModuleGenerator construction....86
14.3 Sub-generators....87
14.4 Master adapter (AdapterGenerator)....88
14.5 Slave adapter (SlaveAdapterGenerator)....89
14.6 Crossbar (CrossbarGenerator)....90
15 Filelist and Test Generation....91
15.1 Filelist generation....92
15.1.1 Filelist sections....92
15.2 Test generation....93
16 Directory Layout....94
16.1 Variant directories....95
17 Package File....96
17.1 What it contains....97
17.2 Excerpt from a generated package....98
17.3 Why it matters....99
18 Top Module....100

18.1 Module signature....101
18.2 Internal structure....102
18.3 Excerpt: subtractive catch-all instantiation....103
18.4 Excerpt: adapter wiring pattern....104
19 Adapters and Crossbar....105
19.1 Master adapter (<name>_adapter.sv)....105
19.1.1 Per-width paths....105
19.2 Slave adapter (<name>_adapter.sv)....106
19.2.1 Response routing....106
19.3 Crossbar (<bridge_name>_xbar.sv)....107
20 Filelist....108
20.1 Path convention....109
20.2 Example filelist head....110
20.3 Filelist is generated, not hand-maintained....111
21 Generated Tests....112
21.1 TB class....113
21.2 Test runner....114
21.3 Monitor stress test....115
21.4 Regeneration discipline for tests....116
22 Full Regeneration Rule....117
22.1 Why partial regeneration is dangerous....118
22.2 Required workflow....119
22.3 Code changes that trigger full regeneration....120
22.4 Files that are never regenerated....121

22.5 Version-mismatch symptoms....122
23 Generated Examples....123
23.1 Representative fixtures....124
23.2 How to verify a regeneration....125
23.3 Fixture as documentation....126
24 DV Suite....127
24.1 Test structure....128
24.2 Running tests....129
24.3 Test levels....130
24.4 Verification coverage....131
24.5 When generator changes affect DV....132
25 Lint and Synthesis Checks....133
25.1 Verilator lint....134
25.2 Out-of-context synthesis....135
25.3 When to run checks....136
26 Adding a Protocol....137
26.1 Step 1: Add the protocol to the validator....138
26.2 Step 2: Add converter/wrapper selection....139
26.3 Step 3: Update the filelist....140
26.4 Step 4: Add a representative fixture....141
26.5 Step 5: Run the full regeneration and test....142
26.6 Example: how APB was added....143
27 Adding a Bridge-Level Option....144
27.1 Step 1: Add the dataclass field....145

27.2 Step 2: Add the TOML loader mapping....146
27.3 Step 3: Validate if necessary....147
27.4 Step 4: Thread it into the emitter....148
27.5 Step 5: Use the option in BridgeModuleGenerator....149
27.6 Step 6: Document and test....150
27.7 Example: xbar_pipeline....151
28 Adding a Port-Level Option....152
28.1 Step 1: Add the PortSpec field....153
28.2 Step 2: Load it from TOML....154
28.3 Step 3: Validate it....155
28.4 Step 4: Thread it into MasterConfig / SlaveInfo....156
28.5 Step 5: Use it in the adapter generator....157
28.6 Step 6: Update the filelist if needed....158
28.7 Example: cdc....159
29 Testing an Extension....160
29.1 Minimum test plan....161
29.2 Regression fixtures....162
29.3 Documentation....163

List of Figures

Figure 3.1: Bridge Generator Pipeline.....71

List of Tables

Table 1: Document information....12
Table 2: Revision history....13
Table 3: Generated Artifact Family....19
Table 4: Identifying Generated Files....26
Table 5: Single-Bridge Mode....28
Table 6: Makefile Targets....30
Table 7: [bridge] scalar keys....33
Table 8: [bridge.clock] table....34
Table 9: Crossbar and arbitration keys....35
Table 10: Monitor and cfg subsystem keys....37
Table 11: [bridge.mon_group] table....39
Table 12: Common port fields....42
Table 13: Slave-only fields....44
Table 14: Optional behavior fields....45
Table 15: Channel semantics....48
Table 16: interface.type values....51
Table 17: interface.skid_depths table....52
Table 18: latency modes....53
Table 19: Bridge-level monitor keys....57
Table 20: Per-port monitor keys....57
Table 21: Per-port cfg fields....59
Table 22: Key files by stage....72
Table 23: Examples of validation failures....82
Table 24: Translation to emitter structs....85
Table 25: Sub-generators....87
Table 26: Test generation....93
Table 27: Representative fixtures....124
Table 28: Test levels....130

1 Bridge Gas Index

Generated: 2026-10-15

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Document Information

2.1 Bridge Generator Architecture Specification

Document information

Property	Value
Document Title	Bridge Generator Architecture Specification
Version	1.1
Date	2026-10-15
Status	Released
Classification	Open Source - MIT License

2.2 Revision History

Revision history

Version	Date	Author	Description
1.0	2026-10-12	RTL Design Sherpa	Initial GAS release — generator code paths verified against `bridge_generator.py`, `bridge_pkg/config.py`, `bridge_pkg/config_validator.py`, `bridge_pkg/config_loader.py`, and emitted artifacts in `rtl/generated/bridge_mix_e/`.
1.1 (renumbered)	2026-10-15	RTL Design Sherpa	The bridge document set (GAS, HAS, MAS) is renumbered to one common rev, 1.1; rows above keep the numbers they were published under. Content: the one-target-per-master limitation is fixed (GitHub #119 L1) – per-ID target tracking (`[bridge].id_track_bits`), B/R response arbiters per master width path, the crossed-W hold; the generated TB carries its connectivity map and the generator emits the AXIL narrow-to-wide alignment test (GitHub #119 L8); per-port outstanding limits, `max_outstanding` (power of

Version	Date	Author	Description
			two 2..64, default 16), size every tracker – the fixed 16 removed (GitHub #119 L5); per-port `latency` modes (`cut_all` / `cut_ax` / `cut_none`) and honoured TOML skid depths (GitHub #119 L7); read-return-atomic and trace-echo ports no longer limited to one target (GitHub #119 L1b).

2.3 Document Purpose

This Generator Architecture Specification (GAS) is the manual for the Bridge **code generator**. It is written for the engineer who must:

- re-run the generator after a bugfix,
- add a new protocol or port option,
- debug a generated module,
- or audit why a particular file was emitted.

It covers the full generator surface: CLI flags, TOML keys, Python data classes, validation rules, emission stages, output files, and the extension points that are safe to touch.

For the hardware that the generator produces, see the **Bridge HAS** and **Bridge MAS**. For product requirements, see **PRD.md**.

2.4 Intended Audience

- RTL engineers modifying or extending the generator
- Verification engineers regenerating fixtures after a generator change
- System architects adding new bridge configurations to the batch
- Integrators debugging a generated bridge top

2.5 Related Documents

- **Bridge HAS** - Hardware Architecture Specification
- **Bridge MAS** - Micro-Architecture Specification
- **PRD.md** - Product requirements and overview
- **CLAUDE.md** - AI development guide and subsystem rules

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

3 What the Generator Produces

The Bridge generator (`bin/bridge_generator.py`) emits a complete, synthesizable AXI4/AXI5 crossbar from a TOML port description and a CSV connectivity matrix. The output is **not** a parameterized Verilog template: every width, count, and protocol conversion is resolved at generation time, and the top module exposes no parameters.

3.1 Generated Artifact Family

For each requested variant (`no` or `mon`), the generator writes a directory under `rtl/generated/<bridge_name>/` containing:

Generated Artifact Family

File	Stamped generator
`<bridge_name>_pkg.sv`	`PackageGenerator`
`<bridge_name>.sv`	`BridgeModuleGenerator`
`<bridge_name>_xbar.sv`	`CrossbarGenerator`
`<master>_adapter.sv`	`AdapterGenerator` (per master)
`<slave>_adapter.sv`	`SlaveAdapterGenerator` (per slave)
`<bridge_name>.f`	`bridge_generator.py` (filelist)
`<bridge_name>.toml`	copied source config
`<bridge_name>_connectivity.csv`	copied connectivity matrix
`__cfg_*_pkg.sv` / `<bridge_name>_cfg.sv`	`CfgRdlGenerator` (only when `use_cfg_regblock = true`)

The header comment in each `.sv` file names the generator class that produced it. This is the fastest way to tell whether a file is generated and which part of the pipeline owns it.

3.2 What Each Variant Means

The TOML `[bridge].variants` list selects which variants to emit:

- `"no"` — bare bridge, no monitor wrapper instances.
- `"mon"` — bridge with `_monlite` wrappers on every enabled port, plus the monbus arbiter and an internal AXIL4/AXIL4 group (unless `internal_axil_group = false`).

Both variants are fully independent; each gets its own directory and its own filelist.

3.3 Scope of Generation

The generator resolves:

- Port prefixes, channel subsets (`rw`/`wr`/`rd`), widths, and protocol variants.
- Width converters (`axi4_dwidth_converter_{wr,rd}`) inserted only where master and slave data widths differ.
- Protocol shims (`axi4_to_apb4_shim`, `axi4_to_apb5_shim`, `axi4_to_axil4_*`, `axi4_to_axil5_*`, `axi4_to_axi3`, `axi4_to_wb4`, AXI5 boundary wrappers) selected by the port `protocol` field.
- The subtractive catch-all slave (`axi4_subtractive_slave`), always present and wired to unmapped addresses.
- Optional monitor wrappers and a cfg register block.

The generator does **not** emit testbench code unless `--generate-tests` is passed.

---

4 When to Regenerate

Any change to the generator or its inputs can change every emitted file. Bridge enforces a **full regeneration discipline**.

4.1 Triggers for Full Regeneration

Regenerate every generated bridge in `rtl/generated/` when any of the following change:

- `bin/bridge_generator.py`
- Any Python file under `bin/bridge_pkg/`
- Any Jinja template under `bin/bridge_pkg/jinja_templates/`
- Any shared converter or wrapper IP the filelist pulls in, if its interface changes

Do **not** regenerate a single bridge and leave the rest untouched. Generated files share package definitions, converter selection logic, and filelist conventions; partial regeneration creates version skew.

4.2 Regeneration Workflow

```
cd projects/components/fabric-gen-ip/bridge/bin
make clean          # removes rtl/generated/
make all            # regenerates every bridge in bridge_batch.csv
```

After regenerating RTL, also regenerate tests if the generator's test templates changed:

```
python bridge_generator.py --bulk bridge_batch.csv --generate-tests
```

Then run the DV suite from a clean build directory:

```
cd ../dv/tests
make clean-all && make run-all-func
```

4.3 When Inputs (Not Code) Change

If only the TOML or CSV for one bridge changes, regenerate that bridge alone is safe only when **no generator code changed**:

```
python bridge_generator.py --ports test_configs/my_bridge.toml
```

If in doubt, run `make clean && make all`.

4.4 Identifying Generated Files

Each generator class stamps its own header. Grepping for the generator name is more reliable than grepping for `bridge_generator.py`:

Identifying Generated Files

Generator class	Header string
`BridgeModuleGenerator`	`Generated by: BridgeModuleGenerator`
`PackageGenerator`	`Generated by: PackageGenerator`
`CrossbarGenerator`	`Generated by: CrossbarGenerator`
`AdapterGenerator`	`Generated by: AdapterGenerator`
`SlaveAdapterGenerator`	`Generated by: SlaveAdapterGenerator`

Hand-written files in `rtl/` (for example `bridge_cam.sv`) are never regenerated and must not be deleted by `make clean`.

5 Generator Entry Points

Bridge has two invocation modes: single-bridge and bulk.

5.1 Single-Bridge Mode

```
cd projects/components/fabric-gen-ip/bridge/bin
python3 bridge_generator.py \
    --ports test_configs/bridge_2x2_rw.toml \
    [--connectivity test_configs/bridge_2x2_rw_connectivity.csv] \
    [--name bridge_2x2_rw] \
    [--output-dir ../rtl/generated]
```

For TOML inputs, `--connectivity` is optional: the loader auto-detects `<bridge_name>_connectivity.csv` next to the TOML file. CSV port files are no longer supported.

CLI flags:

Single-Bridge Mode

Flag	Default	Purpose
`--ports`	required	Path to the TOML port configuration
`--connectivity`	auto	Path to the CSV connectivity matrix
`--name`	from TOML or auto	Output module base name
`--output-dir`	`../rtl/generated`	Where to write the bridge directory
`--generate-tests`	off	Also emit cocotb TB class and test runner
`--output-tb`	`../dv/tbclasses`	TB class output directory
`--output-test`	`../dv/tests`	Test runner output directory

5.2 Bulk Mode

```
python3 bridge_generator.py --bulk bridge_batch.csv
```

The batch CSV columns are:

```
name,ports,connectivity,output_dir,output_tb,output_test
```

Comment lines starting with # are ignored. The `name` and `connectivity` fields may be empty; empty connectivity triggers auto-detection from the TOML filename.

5.3 Makefile Targets

The `bin/Makefile` wraps the bulk path:

Makefile Targets

Target	Action
`make all`	`bridge_generator.py --bulk bridge_batch.csv`
`make clean`	remove `rtl/generated/` and `rtl/filelists/`
`make regen`	alias for `make clean && make all`

5.4 Return Value

`generate_bridge()` returns a tuple `(success: bool, emitted_variants: list[tuple[str, bool]])`. A non-empty `emitted_variants` list means at least one variant was written. The bulk driver exits non-zero if any bridge or test generation fails.

---

6 Bridge-Level TOML Keys

The TOML `[bridge]` table configures the bridge as a whole. It is parsed by `bridge_pkg.config_loader.load_config()` into the `BridgeConfig` dataclass (`bridge_pkg/config.py`).

6.1 [bridge] scalar keys

[bridge] scalar keys

Key	Type	Default	Dataclass field	Description
name	string	""	BridgeConfig.name	Output base name. If empty, the generator auto-names from topology (bridge_<M>x<S>_<channels>).
variants	list of strings	["no"]	BridgeConfig.variants	Build variants to emit. Each entry must be "no" or "mon". At least one entry is required in TOML.
description	string	""	ignored	Human-readable comment; copied into generated header only.

6.2 `[bridge.clock]` table

[bridge.clock] table

Key	Type	Default	Used by	Description
`name`	string	`"aclk"`	header comment	Clock signal name (informational).
`frequency_mhz`	int	none	header comment	Clock frequency (informational).

6.3 Crossbar and arbitration keys

Crossbar and arbitration keys

Key	Type	Default	Dataclass field	Description
`xbar_pipeline`	boolean	`false`	`BridgeConfig.xbar_pipeline`	Add skid register stage on every slave-side channel inside the crossbar.
`arbitration`	string	`"rr"`	`BridgeConfig.arbitration`	Per-slave arbitration policy. `"rr"` = round-robin; `"qos"` = QoS + aging.
`qos_aging_shift`	int	4	`BridgeConfig.qos_aging_shift`	For `"qos"`, a waiting request gains one priority level every `2**shift` cycles.
`id_track_bits`	int 0..8	4	`BridgeConfig.id_track_bits`	Per-ID target table in every master adapter: `2**id_track_bits` entries by the ID's low bits (never more than the port's ID). A request waits only while its ID is in flight to a different slave. Fixes the one-target-per-master limitation (GitHub #119 L1). Out of range is a `ValueError` at

Key	Type	Default	Dataclass field	Description
				load.
`max_outstanding`	int, power of two 2..64	16	`PortSpec.max_outstanding` (default for every port)	Outstanding-transaction limit per direction for every port that does not set its own: master-adapter tracker depth and per-ID count width; slave-adapter `bridge_cam` / in-order FIFO depth; the crossbar's W-destination (per master path) and W-owner (per slave, plus 2 on a registered crossbar) FIFOs follow. Fixes the fixed 16 (GitHub #119 L5).

6.4 Monitor and cfg subsystem keys

Monitor and cfg subsystem keys

Key	Type	Default	Dataclass field	Description
`mon_preset`	string	`"lite"`	`BridgeConfig.mon_preset`	Baseline monitor reporter enables. Accepted aliases: `"all"`, `"error_only"`, `"functional"`, `"none"`, `"lite"`. Since 2026-09-26 every monitored port uses `axi_monitor_lite`, so the preset resolves to the lite set regardless.
`use_all_monitors`	boolean	`false`	`BridgeConfig.use_all_monitors`	Force `USE_MONITOR=1` on every port. Mutually exclusive with `use_no_monitors`.
`use_no_monitors`	boolean	`false`	`BridgeConfig.use_no_monitors`	Force `USE_MONITOR=0` on every port. Mutually exclusive with `use_all_monitors`.
`internal_axil_group`	boolean	`true`	`BridgeConfig.internal_axil_group`	For `"mon"` variants, instantiate the internal `monbus_axil4_axil4_group`. When `false`, expose the

Key	Type	Default	Dataclass field	Description
				aggregated monbus stream at the top.
`use_cfg_regblock`	boolean	`false`	`BridgeConfig.use_cfg_regblock`	Replace the 350+ individual `cfg_*` top-level pins with a PeakRDL-generated cfg register block.

6.5 [bridge.mon_group] table

[bridge.mon_group] table

Key	Type	Default	Dataclass field	Description
slave_protocol	string	"axil4"	MonGroupConfig.slave_protocol	Protocol of the monbus group slave-read port. "axil4" or "axi4". Deprecated spelling "axil" is normalized to "axil4".
master_protocol	string	"axil4"	MonGroupConfig.master_protocol	Protocol of the monbus group master-write port. "axil4" or "axi4".
axi_id_width	int	8	MonGroupConfig.axi_id_width	AXI ID width when either side is axi4.
max_burst_beats	int	64	MonGroupConfig.max_burst_beats	Max write burst when master side is axi4.
fifo_depth_err	int	64	MonGroupConfig.fifo_depth_err	Error FIFO depth.
fifo_depth_write	int	96	MonGroupConfig.fifo_depth_write	Write-data FIFO depth in beats.
flush_timeout_cycles	int	1024	MonGroupConfig.flush_timeout_cycles	Flush timeout.
use_compression	int	0	MonGroupConfig.use_compression	Compression enable.

6.6 [bridge.defaults] table

The `defaults.skid_depths` sub-table provides default skid depths for interface wrappers:

```
[bridge.defaults]
skid_depths = {ar = 2, r = 4, aw = 2, w = 4, b = 2}
```

Per-port `interface.skid_depths` overrides these defaults, and a port's `latency` mode then sets the channels it cuts to 0 (pass-through). Valid per-channel depths are integers in the range `2..8` inclusive. Until GitHub #119 L7 (2026-10-15) both tables were parsed and ignored – every wrapper was built 2/4/2/2/2; they take effect now. Any other key under `[bridge.defaults]` is reported and ignored.

7 Port Specification Schema

Each master and slave is described by a TOML table under `[[bridge.masters]]` or `[[bridge.slaves]]`. The loader converts each table into a `PortSpec` dataclass (`bridge_pkg/config.py`).

7.1 Common port fields

Common port fields

Field	Type	Default	PortSpec attribute	Description
`name`	string	required	`port_name`	Unique identifier used in generated module names and file names.
`prefix`	string	required	`prefix`	Signal prefix. The generator normalizes a missing trailing underscore (`cpu_m_axi` → `cpu_m_axi_`).
`protocol`	string	required	`protocol`	One of `axi3`, `axi4`, `axi5`, `apb`, `apb5`, `axil`, `axil5`, `wb4`.
`channels`	string	required	`channels`	One of `rw`, `wr`, `rd`. Slaves must declare what they support; the generator trims unneeded channels.
`data_width`	int	required	`data_width`	Data bus width in bits.
`addr_width`	int	required	`addr_width`	Address bus width in bits.
`id_width`	int	`0`	`id_width`	AXI ID width. Required > 0 for `axi4`/`axi5` masters; must be 0 for APB,

Field	Type	Default	PortSpec attribute	Description
				AXI-Lite, and Wishbone.
`user_width`	int	`0`	(passed to wrapper)	AXI4 user signal width.

7.2 Slave-only fields

Slave-only fields

Field	Type	Default	`PortSpec` attribute	Description
`base_addr`	hex string/ int	required for slaves	`base_addr`	Start of the slave address window. Must be 4 KiB-aligned.
`addr_range`	hex string/ int	required for slaves	`addr_range`	Size of the slave window. Must be a non-zero multiple of 4 KiB.

7.3 Optional behavior fields

Optional behavior fields

Field	Type	Default	PortSpec attribute	Description
`interface`	table	none	see below	Timing-isolation wrapper configuration.
`cdc`	boolean	`false`	`cdc`	Run this slave port on its own clock behind `axi4_cdc_{wr,rd}`. Legal only for `protocol="axi4"` slave ports.
`enable_ooo`	boolean	`false`	`enable_ooo`	Mark slave as out-of-order capable. (No generated fixture currently uses this.)
`max_outstanding`	int, power of two 2..64	`[bridge].max_outstanding` (16)	`max_outstanding`	This port's outstanding limit per direction (master: tracker depth; slave: response-tracker depth). The internal subtractive slave stays at 16.
`use_monitor`	boolean	`true`	`use_monitor`	Per-port monitor enable; meaningful only for `"mon"`

Field	Type	Default	PortSpec attribute	Description
				variants.
`mon_add`	list of strings	`[]`	`mon_add`	Additive reporter cones beyond the preset. Names come from `MON_CONES`: `error`, `timeout`, `compl`, `threshold`, `perf`, `debug`.
`mon_remove`	list of strings	`[]`	`mon_remove`	Reporter cones to remove from the preset.
`axi5_features`	list of strings	`[]`	`axi5_features`	AXI5 sideband feature list. Legal only on `axi5` or `axil5` ports.
`internal`	boolean	`false`	`internal`	Port has no top-level pins. Used automatically for the subtractive catch-all.

7.4 axi5_features values

The validator (bridge_pkg/config_validator.py) accepts:

- Droppable sideband: nsaid, trace, mpam, mecid, unique, chunking.
- Connectivity-gated (every connected path must be native AXI5, same feature, same width): poison, atomic, mte.

On axil5 ports, only user and exclusive are forwardable; trace, loop, mpam, mecid, nsaid, poison are rejected because they have no AXI4 source/destination.

7.5 Channel semantics

Channel semantics

`channels`	Master gets	Slave must support
`rw`	AW, W, B, AR, R	AW, W, B, AR, R
`wr`	AW, W, B only	AW, W, B
`rd`	AR, R only	AR, R

A `wr` master may connect to a `wr` or `rw` slave; a `rd` master may connect to a `rd` or `rw` slave. A `wr` master cannot connect to a `rd` slave, and vice versa.

7.6 Example

```
[[bridge.masters]]
name       = "cpu"
prefix     = "cpu_m_axi_"
protocol   = "axi4"
channels   = "rw"
id_width   = 4
addr_width = 32
data_width = 64
user_width = 1

[[bridge.slaves]]
name       = "ddr"
prefix     = "ddr_s_axi_"
protocol   = "axi4"
channels   = "rw"
id_width   = 6
addr_width = 32
data_width = 512
user_width = 1
base_addr  = "0x00000000"
addr_range = "0x80000000"
```

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

8 Interface Wrapper Configuration

The optional per-port `interface` table selects a timing-isolation wrapper and its skid depths. If omitted, the port is connected directly to the crossbar (no wrapper).

8.1 `interface.type` values

interface.type values

Value	Wrapper instantiated
`axi4_master`	`axi4_slave_wr` / `axi4_slave_rd` (master port acts as AXI slave to external master)
`axi4_slave`	`axi4_master_wr` / `axi4_master_rd` (slave port acts as AXI master to external slave)
`axi4_master_mon`	`_monlite` variant of the master wrapper
`axi4_slave_mon`	`_monlite` variant of the slave wrapper

The `_monlite` wrappers are used unconditionally since 2026-09-26; the old non-lite wrappers were retired.

8.2 `interface.skid_depths` table

interface.skid_depths table

Key	Type	Default	Valid range	Description
`ar`	int	from `[bridge.defaults.skid_depths]`	2..8	Read address skid depth.
`r`	int	from defaults	2..8	Read data skid depth.
`aw`	int	from defaults	2..8	Write address skid depth.
`w`	int	from defaults	2..8	Write data skid depth.
`b`	int	from defaults	2..8	Write response skid depth.

8.3 `latency` (port key)

`latency` selects which channels of the port's timing wrapper are registered (GitHub #119 L7). A cut channel's skid depth becomes 0, which `gaxi_skid_buffer` builds as a combinational pass-through.

latency modes

Value	Registered channels	Port adds
`cut_all` (default)	AW, W, B, AR, R	1 cycle per direction
`cut_ax`	AW, AR	1 cycle on requests, 0 on W / B / R
`cut_none`	none	0 cycles: the port boundary is wires

Legal only on a same-clock `axi4` / `axi5` port; a converted port (AXI-Lite, APB, Wishbone, AXI3) or a `cdc = true` port has buffers of its own that the mode does not reach, so the loader rejects it there. The master adapter keeps the resolved depths as module parameters (`SKID_DEPTH_*`), so a single generated adapter can still be re-timed by overriding them at elaboration. Measured on `bridge_2x2_rw_lat` by `test_bridge_2x2_rw_lat_modes`.

Example:

```
[[bridge.masters]]
name = "dma"
prefix = "dma_m_axi_"
protocol = "axi4"
channels = "rw"
id_width = 4
addr_width = 32
data_width = 512
user_width = 1
interface = {type = "axi4_master", skid_depths = {ar = 4, r = 8, aw =
4, w = 8, b = 4}}
```

8.4 How the wrapper selection flows

1. `bridge_pkg/config_loader.py` reads the `interface` table into `PortSpec` attributes.
2. `bridge_generator.py` passes the port protocol and `use_monitor` flag to `BridgeModuleGenerator`.
3. `AdapterGenerator` (`bridge_pkg/generators/adapter_generator.py`) and `SlaveAdapterGenerator` (`bridge_pkg/generators/slave_adapter_generator.py`) instantiate the appropriate wrapper module based on protocol and monitor state.
4. The generated adapter module wires the wrapper's `fub_axi_*` internal face to the crossbar.

8.5 Protocol-specific wrapper notes

- `axi5` ports use `axi5_slave_{wr,rd}` on master sides and `axi5_master_{wr,rd}` on slave sides.
- `apb`/`apb5` master ports use `apb{4,5}_to_axi4` front-end converters inside the adapter.
- `wb4` master ports use `wb4_to_axi4`.
- `axi3` master ports use `axi3_to_axi4`.
- `apb`, `apb5`, `axil`, `axil5`, `wb4`, and `axi3` slave ports use the corresponding completer shim (`axi4_to_apb4_shim`, `axi4_to_wb4`, etc.).

9 Monitor and CFG Subsystem Keys

Bridge can optionally emit a monitor infrastructure and a cfg register block. This section documents the keys that control those subsystems.

9.1 Monitor subsystem

The "mon" variant wraps each enabled port with a _monlite timing wrapper. The wrapper exposes a monbus_* packet interface that feeds a per-bridge monbus_arbiter and, by default, a monbus_axil4_axil4_group.

9.1.1 Bridge-level monitor keys

Bridge-level monitor keys

Key	Type	Default	Description
mon_preset	string	"lite"	Baseline reporter enables. Since 2026-09-26 all monitored ports use axi_monitor_lite, so every preset resolves to the same hardware; the field is kept for backward-compatible config loading.
use_all_monitors	boolean	false	Override every port's use_monitor to true.
use_no_monitors	boolean	false	Override every port's use_monitor to false. Mutually exclusive with use_all_monitors.

9.1.2 Per-port monitor keys

Per-port monitor keys

Key	Type	Default	Description
use_monitor	boolean	true	Include the _monlite wrapper on this port in a "mon" variant.
mon_add	list	[]	Extra reporter cones to enable beyond the preset.
mon_remove	list	[]	Reporter cones to disable from the preset.

MON_CONES in bridge_pkg/config.py defines the valid cone names: error, timeout, compl, threshold, perf, debug.

9.1.3 `[bridge.mon_group]` table

This table selects the internal monbus aggregation module. See “Bridge-Level TOML Keys” for the full schema. The selected module is:

```
f"monbus_{slave_protocol}_{master_protocol}_group"
```

Examples: `monbus_axil4_axil4_group`, `monbus_axi4_axi4_group`.

When `internal_axil_group = false`, the bridge still instantiates the `monbus_arbiter` but exposes its aggregated stream as `monbus_agg_*` top-level outputs, letting an external consumer provide the AXIL group.

9.2 CFG register block

Setting `use_cfg_regblock = true` replaces the 350+ individual `cfg_<port>_<idx>_<chan>_*` top-level pins with a PeakRDL-generated register block.

9.2.1 What changes

- `CfgRdlGenerator` (`bridge_pkg/generators/cfg_rdl_generator.py`) builds an RDL schema from the adapter list and `BridgeModuleGenerator._MON_GROUP_CFG`.
- PeakRDL emits `<bridge_name>_cfg_pkg.sv` and `<bridge_name>_cfg.sv` into the bridge directory.
- `bridge_generator.py` patches the top module to narrow the 32-bit `s_cfg_axil_*addr` into the regblock's actual address width.
- The filelist gains the cfg package and body, plus a Verilator waiver for PeakRDL's `field_combo MULTIDRIVEN` warning.

9.2.2 Per-port cfg fields

`CfgAdapterPort` is constructed in `bridge_generator.py` for every master and every non-internal slave. Fields include:

Per-port cfg fields

Field	Meaning
`name`	Port name
`idx`	Index within direction
`has_wr` / `has_rd`	Which directions exist after channel trimming
`perfmon_enabled`	Whether perf counters exist for this port
`n_addr_ranges`	Number of address ranges (currently 0)
`addr_width`	Address width for this adapter

The subtractive catch-all slave is deliberately excluded from the cfg map so that adding or removing it does not shift register offsets.

10 Validation Rules

Configuration validation is centralized in `bridge_pkg/config_validator.py`. The loader calls `validate_config(masters, slaves, connectivity)` before any RTL is emitted. Validation failures raise `ValidationError` and stop generation.

10.1 Field validation (`validate_required_fields`)

- Every port must have `name` and `prefix`.
- `data_width` and `addr_width` must be positive.
- `axi4` masters must have `id_width > 0`.
- `axil`/`axil5` masters must have `id_width == 0`.
- `apb`/`apb5`/`wb4`/`axi3` masters must have `addr_width == 32`.
- Slaves must have `base_addr` and a positive `addr_range`.
- Slave `base_addr` and `addr_range` must be 4 KiB-aligned / a multiple of 4 KiB.

10.2 Channel and protocol validation

- `channels` must be one of `rw`, `wr`, `rd` (`validate_channels`).
- `protocol` must be one of `axi3`, `axi4`, `axi5`, `apb`, `apb5`, `axil`, `axil5`, `wb4` (`validate_protocol`).
- Slaves must explicitly specify `channels` (`validate_slave_channels_explicit`).

10.3 Protocol-specific constraints

10.3.1 APB / APB5 (`validate_apb_constraints`)

- `channels` must be `rw`.
- `data_width` must be 8, 16, or 32.
- `id_width` must be 0.

10.3.2 Wishbone B4 (`validate_wb4_constraints`)

- `channels` must be `rw`.
- `data_width` must be 8, 16, 32, or 64.
- `id_width` must be 0.

10.3.3 AXI3 (`validate_axi3`)

- AXI3 masters must have `id_width > 0`.
- `data_width` must be 32, 64, 128, or 256.
- CDC is not supported.

10.3.4 AXI5 (`validate_axi5`)

- `axi5_features` may appear only on `axi5` or `axil5` ports.
- Droppable features: `nsaid`, `trace`, `mpam`, `mecid`, `unique`, `chunking`.
- Connectivity-gated features: `poison`, `atomic`, `mte`.
- `mte` and `chunking` require `data_width >= 128` (`validate_axi5_wide_features`).
- `poison`, `atomic`, `mte` require every connected path to be native AXI5 with the feature enabled and the same data width (`validate_axi5_poison_connectivity`).
- An `rw` AXI5 master with `atomic` may only connect to `rw` AXI5 slaves (`validate_axi5_atomic_read_return`).
- `axil5` ports accept only `user` and `exclusive` as forwardable features (`validate_axil5_features`).

10.4 Master-slave compatibility (`validate_master_slave_compatibility`)

- A `wr` master cannot connect to a `rd` slave.
- A `rd` master cannot connect to a `wr` slave.
- Compatibility is checked only for connected pairs (per the CSV connectivity matrix).

10.5 Address map validation (`validate_address_map`)

- Positively-decoded slave windows must not overlap.
- The subtractive catch-all (`internal=true`) is excluded from overlap checks.

10.6 Slave ID width (`validate_slave_id_widths`)

With more than one master, every AXI slave port's ID is widened by `$clog2(NUM_MASTERS)` master-index bits. The validator checks that each slave's declared `id_width` is at least:

```
widest_master_id_width + clog2(num_masters)
```

10.7 CDC constraints (`validate_cdc_constraints`)

- `cdc=true` is legal only on external AXI4 slave ports.
- Internal slaves cannot be CDC ports.

10.8 Warning for dropped sideband (`warn_axi5_dropped_sideband`)

Droppable AXI5 sideband that terminates mid-path prints a generation-time warning naming the path and the reason (non-AXI5 peer, missing feature, or width converter).

11 Pipeline Overview

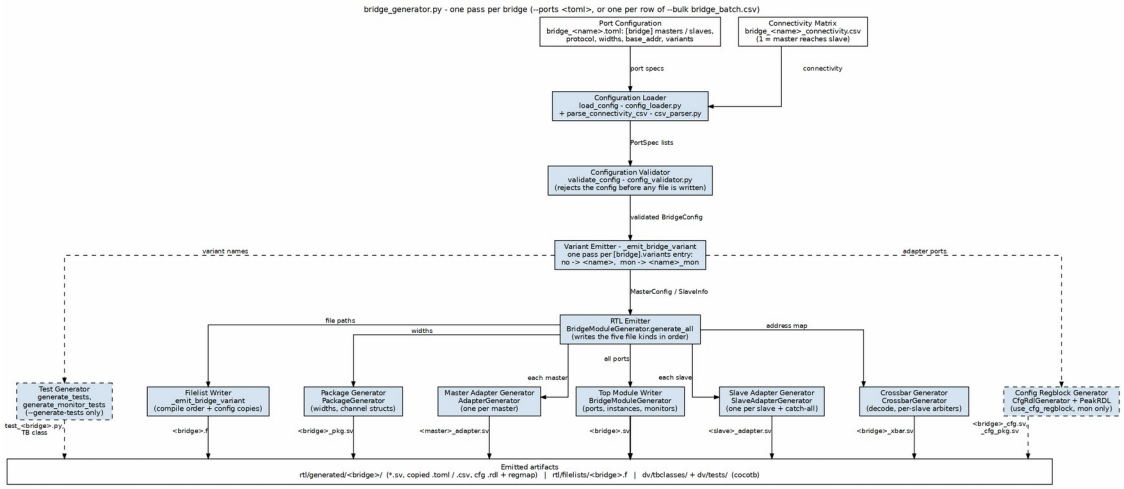
The Bridge generation pipeline has five stages:

1. **Parse** — TOML and CSV files are read into Python data structures.
2. **Validate** — `validate_config()` checks semantic and protocol rules.
3. **Translate** — `PortSpec` objects are converted to `MasterConfig`/`SlaveInfo` objects consumed by the emitter.
4. **Emit RTL** — `BridgeModuleGenerator.generate_all()` writes package, adapters, crossbar, and top module.
5. **Emit collateral** — filelist, copied configs, optional regblock, optional tests.

Each variant (`no`, `mon`) runs stages 3–5 independently and writes to its own directory.

11.1 Pipeline diagram

11.1.1 Figure 3.1: Bridge Generator Pipeline



Bridge generator pipeline

Configuration enters at the top and passes down the center column: load, validate, then one pass per variant into the RTL emitter, which fans out to one sub-generator per file kind. Every output lands on the emitted-artifacts bar at the bottom. The dashed boxes run only when asked for: the cocotb test generator under `--generate-tests`, and the PeakRDL config regblock for a `use_cfg_regblock` bridge's `mon` variant. Nothing in the generator arbitrates, so the figure has no trapezium.

11.2 Key files by stage

Key files by stage

Stage	File	Function
Parse	`bridge_pkg/config_loader.py`	`load_config()` reads TOML and CSV.
Validate	`bridge_pkg/config_validator.py`	`validate_config()` runs all checks.
Translate	`bridge_generator.py`	`_emit_bridge_variant()` builds `MasterConfig`/`SlaveInfo`.
Emit RTL	`bridge_pkg/components/bridge_module_generator.py`	`BridgeModuleGenerator.generate_all()` orchestrates sub-generators.
Emit adapters	`bridge_pkg/generators/adapter_generator.py`	`AdapterGenerator` emits master adapters.
Emit slave adapters	`bridge_pkg/generators/slave_adapter_generator.py`	`SlaveAdapterGenerator` emits slave adapters.
Emit crossbar	`bridge_pkg/generators/crossbar_generator.py`	`CrossbarGenerator` emits the internal crossbar.
Emit package	`bridge_pkg/generators/package_generator.py`	`PackageGenerator` emits the SV package.
Emit filelist	`bridge_generator.py`	`_emit_bridge_variant()` writes the `.f` filelist.
Emit tests	`bridge_generator.py`	`generate_tests()` / `generate_monitor_tests()` write cocotb files.
Emit cfg	`bridge_pkg/generators/cfg_rdl_generator.py`	`CfgRdlGenerator` emits the PeakRDL register block.

12 Configuration Loading

Configuration loading is implemented in `bridge_pkg/config_loader.py`. The entry point is `load_config(ports_file, connectivity_file=None)`.

12.1 TOML parsing

The loader uses Python's `tomllib` (or `tomli` on older Pythons) to read the TOML file. It expects:

- A `[bridge]` table with scalar keys and `[[bridge.masters]]` / `[[bridge.slaves]]` arrays.
- An optional `[bridge.mon_group]` table.
- An optional `[bridge.defaults]` table with a `skid_depths` sub-table.

Hex strings such as `"0x10000000"` are accepted for `base_addr` and `addr_range`.

12.2 Connectivity CSV

The connectivity matrix is a CSV with the first column labeled `master\slave` and subsequent columns named after slaves. Example:

```
master\slave,ddr,sram,apb_periph
cpu,1,1,0
dma,1,0,1
```

If `connectivity_file` is `None`, `find_connectivity_csv(ports_file)` searches for `<stem>_connectivity.csv` next to the TOML file.

The loader builds a dictionary `connectivity[master_name] = set(slave_names)`.

12.3 `BridgeConfig` construction

`load_config()` returns a `BridgeConfig` dataclass (`bridge_pkg/config.py`) populated with:

- `masters`: list of `PortSpec` (direction="master")
- `slaves`: list of `PortSpec` (direction="slave")
- `connectivity`: dict mapping master names to sets of slave names
- bridge-level fields such as `variants`, `xbar_pipeline`, `arbitration`, `mon_preset`, `use_cfg_regblock`, `mon_group`

The subtractive catch-all slave is **not** added by the loader; `BridgeModuleGenerator` appends it internally during emission.

12.4 Normalization

- Port prefixes are **not** normalized by the loader; normalization happens later in `bridge_generator.py` via `_normalize_prefix()`.
- `MonGroupConfig` normalizes the deprecated `"axil"` spelling to `"axil4"` in `__post_init__`.
- `dest_width` and `id_width` are not relevant to Bridge; those are Delta-specific.

13 Validation

Validation runs immediately after loading. The function `validate_config(masters, slaves, connectivity)` in `bridge_pkg/config_validator.py` is the single gatekeeper.

13.1 Validation order

1. **Per-port required fields** — `validate_required_fields(port)` for every master and slave.
2. **Channel and protocol values** — `validate_channels()` and `validate_protocol()`.
3. **Protocol constraints** — APB, WB4, AXI3, CDC.
4. **AXI5 feature validation** — `validate_axi5()`, wide-feature checks, poison connectivity, atomic read-return rules, axil5 feature checks.
5. **Slave ID width** — `validate_slave_id_widths()`.
6. **Address map** — `validate_address_map()`.
7. **Master-slave compatibility** — `validate_master_slave_compatibility()`.
8. **Dropped-sideband warnings** — `warn_axi5_dropped_sideband()` (non-fatal).

13.2 Error behavior

Every validator raises `ValidationError(message)` on failure. `bridge_generator.py` catches the exception, prints the traceback, and returns `(False, [])` so the bulk driver can count the failure.

13.3 Examples of validation failures

Examples of validation failures

Scenario	Validator	Message highlights
`channels = "rx"`	`validate_channels`	`Invalid channels 'rx'... Must be one of {'rw', 'rd', 'wr'}`
`protocol = "axi6"`	`validate_protocol`	`Invalid protocol 'axi6'... Must be one of {'axi3', 'axi4', 'axi5', 'apb', 'apb5', 'axil', 'axil5', 'wb4'}`
APB with `id_width = 4`	`validate_apb_constraints`	`must have id_width=0`
AXI5 `poison` on a path to an AXI4 slave	`validate_axi5_poison_connectivity`	`connected slave cannot carry it natively`
Overlapping slave windows	`validate_address_map`	`Address overlap detected`
`wr` master to `rd` slave	`validate_master_slave_compatibility`	`cannot connect to read-only slave`

13.4 Adding a new validation rule

To add a rule:

1. Write a validator function that accepts `(masters, slaves, connectivity)` or a single `PortSpec`.
2. Call it from `validate_config()` at the appropriate point in the order above.
3. Update this chapter and the configuration chapter with the new rule.

Keep validators stateless and raise `ValidationError` only; do not mutate the config.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

14 Adapter and Crossbar Emission

After validation, `bridge_generator.py` calls `_emit_bridge_variant()` for each variant in `BridgeConfig.variants`. That function translates `PortSpec` objects into `MasterConfig`/`SlaveInfo` objects and drives `BridgeModuleGenerator`.

14.1 Translation to emitter structs

`MasterConfig` and `SlaveInfo` are lightweight structs consumed by the generators. Key fields:

Translation to emitter structs

Field	`MasterConfig`	`SlaveInfo`
`name`	port name	port name
`prefix`	normalized prefix	normalized prefix
`data_width`	int	int
`addr_width`	int	int
`id_width`	int	int
`channels`	`rw`/`wr`/`rd`	`rw`/`wr`/`rd`
`slave_connections`	list of slave indices	—
`protocol`	string	string
`use_monitor`	bool	bool
`mon_enables`	dict	dict
`axi5_features`	list	list
`cdc`	—	bool
`enable_ooo`	—	bool
`internal`	—	bool

14.2 `BridgeModuleGenerator` construction

`_emit_bridge_variant()` instantiates `BridgeModuleGenerator` with:

```
BridgeModuleGenerator(
    bridge_name=output_name,
    enable_monitoring=use_monitor,
    internal_axil_group=internal_axil_group,
    use_all_monitors=...,
    use_no_monitors=...,
    use_cfg_regblock=...,
    mon_group=...,
    xbar_pipeline=...,
    arbitration=...,
    qos_aging_shift=...,
    id_track_bits=...,
)
```

Then it calls `gen.add_master(master)` and `gen.add_slave(slave)` for every port, and finally `gen.generate_all(bridge_dir)`.

14.3 Sub-generators

`BridgeModuleGenerator.generate_all()` returns a dict mapping a tag to a file path. The tags are:

Sub-generators

Tag	Generator class	Output file
`package`	`PackageGenerator`	`<bridge>_pkg.sv`
`adapter_<name>`	`AdapterGenerator`	`<name>_adapter.sv`
`slave_adapter_<name>`	`SlaveAdapterGenerator`	`<name>_adapter.sv`
`bridge`	`BridgeModuleGenerator` itself	`<bridge>.sv`
`crossbar`	`CrossbarGenerator`	`<bridge>_xbar.sv`

The package is always emitted first so later modules can `import` it. Adapter file names are based on the port `name`, not the prefix.

14.4 Master adapter (`AdapterGenerator`)

The master adapter:

1. Declares the external port with the configured prefix and channel subset.
2. Optionally instantiates a timing wrapper (`axi4_slave_wr/rd`, `_monlite` variants, or AXI5 boundary wrappers).
3. Optionally instantiates a protocol converter (`apb4_to_axi4`, `axi3_to_axi4`, `wb4_to_axi4`, `axi5_atomic_filter`, etc.).
4. Generates per-width output paths (32b, 64b, 128b, 256b, 512b) and inserts `axi4_dwidth_converter_{wr,rd}` or `axil_to_axi4_wide_align_{wr,rd}` only where needed.
5. Emits address decode logic producing `<name>_slave_select_aw` / `ar` and a `bridge_id` sideband.

14.5 Slave adapter (`SlaveAdapterGenerator`)

The slave adapter:

1. Declares the external slave port with the configured prefix and protocol.
2. Optionally instantiates a timing wrapper or protocol shim (`axi4_to_apb4_shim`, `axi4_to_axi3`, `axi4_to_wb4`, AXI5 boundary wrappers).
3. Carries the `bridge_id` response tracker: `bridge_cam` for an AXI4/AXI5 slave of a multi-master bridge or an `enable_ooo` slave (`SlaveAdapterGenerator.use_cam`), an in-order FIFO otherwise.
4. Handles CDC if `cdc=true` by inserting `axi4_cdc_{wr,rd}`.

14.6 Crossbar (`CrossbarGenerator`)

The crossbar module wires all master-adapter outputs to all slave-adapter inputs. It contains:

- AW and AR request matrices.
- Per-slave round-robin arbiters (or QoS arbiters when `arbitration="qos"`).
- W-owner FIFOs and response routing by `bridge_id`.
- Optional registered crossbar stage when `xbar_pipeline=true`.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

15 Filelist and Test Generation

After RTL emission, `_emit_bridge_variant()` writes the filelist and optionally generates cocotb tests.

15.1 Filelist generation

The filelist is written to `rtl/filelists/<bridge_name>.f`, two levels above the bridge directory. It is a complete compile closure using `$REPO_ROOT`-relative paths.

15.1.1 Filelist sections

1. **Include directory** — `+incdir+$REPO_ROOT/rtl/amba/includes`.
2. **Reset macro header** — `-f $REPO_ROOT/rtl/amba/filelists/reset_defs.f`.
3. **Monitor packages** — `-f $REPO_ROOT/rtl/amba/filelists/monitor_pkgs.f` (always included, even for `"no"` variants, because the subtractive catch-all references them).
4. **Subtractive catch-all** — `-f $REPO_ROOT/rtl/amba/filelists/axi4_subtractive_slave.f`.
5. **Generated files** — package first, then other generated files sorted by tag.
6. **AXI4/AXI5 wrapper closures** — `axi4_slave_wr.f`, `axi4_slave_rd.f`, `axi4_master_wr.f`, `axi4_master_rd.f`, plus AXI5 variants when needed.
7. **Converter closures** — `axi_data_upsize.f`, `axi_data_dnsize.f`, `axi4_dwidth_converter_{wr,rd}.f`, `axil_to_axi4_wide_align_{wr,rd}.f`.
8. **Protocol shim closures** — APB, AXI-Lite, AXI3, Wishbone, AXI5 atomic filters/read-return trackers as needed.
9. **CDC closures** — `axi4_cdc_wr.f`, `axi4_cdc_rd.f` if any slave has `cdc=true`.
10. **Monitor infrastructure** — `_monlite` wrapper filelists and `monbus_arbiter.f`, plus the selected `monbus_<p1>_<p2>_group.f` for `"mon"` variants.
11. **Verilator waiver** — `bridge_regblock.vlt` when a cfg regblock is present.

The filelist is generated by string construction in `bridge_generator.py`, not by a template. Every dependency is pulled in via the owning component's closure filelist rather than by hand-listing individual `.sv` files.

15.2 Test generation

When `--generate-tests` is passed:

- For `"no"` variants, `generate_tests()` emits a TB class in `dv/tbclasses/` and a test runner in `dv/tests/`.
- For `"mon"` variants, `generate_monitor_tests()` emits a TB class and a monitor-stress test runner.

Both functions use Jinja2 templates from `bin/bridge_pkg/jinja_templates/`:

Test generation

Template	Output
`bridge_tb_class.py.j2`	`dv/tbclasses/<class_module>.py`
`bridge_test_file.py.j2`	`dv/tests/test_<bridge>.py`
`bridge_monitor_test.py.j2`	`dv/tests/test_<bridge>_monitor.py`

The TB class name is derived from the bridge name by `build_tb_class_name()`. The import block is selected by `_tb_import_block()`; if the output path is not dotted-importable (for example, because of the hyphen in `fabric-gen-ip`), the generated test falls back to a `sys.path` insertion.

16 Directory Layout

For each variant the generator creates a directory under the requested output dir:

```
rtl/generated/<bridge_name>/
├── <bridge_name>_pkg.sv
├── <bridge_name>.sv
├── <bridge_name>_xbar.sv
├── <master0>_adapter.sv
├── <master1>_adapter.sv
├── ...
├── <slave0>_adapter.sv
├── <slave1>_adapter.sv
├── ...
├── <bridge_name>.toml              (copied source config)
├── <bridge_name>_connectivity.csv  (copied connectivity matrix)
├── <bridge_name>_cfg_pkg.sv        (when use_cfg_regblock=true)
└── <bridge_name>_cfg.sv            (when use_cfg_regblock=true)

rtl/filelists/
└── <bridge_name>.f
```

If `--generate-tests` is used, the following are also written:

```
dv/tbclasses/
└── <tb_class_module>.py
dv/tests/
└── test_<bridge_name>.py
```

For "mon" variants the test runner is named `test_<bridge_name>_monitor.py`.

16.1 Variant directories

The "no" variant writes to <base_name>/. The "mon" variant writes to <base_name>_mon/.

Example for a config with name = "bridge_mix_e" and variants = ["no"]:

```
rtl/generated/bridge_mix_e/
├── bridge_mix_e_pkg.sv
├── bridge_mix_e.sv
├── bridge_mix_e_xbar.sv
├── cpu_axi3_adapter.sv
├── dma_axi4_adapter.sv
├── host_apb_adapter.sv
├── ddr_axi3_adapter.sv
├── sram_axi4_adapter.sv
├── apb_periph_adapter.sv
├── subtractive_adapter.sv
├── bridge_mix_e.toml
└── bridge_mix_e_connectivity.csv
```


---

17 Package File

The package file (`<bridge_name>_pkg.sv`) is generated by `PackageGenerator` and is always the first file compiled.

17.1 What it contains

- `BRIDGE_ID_WIDTH` — width of the internal `bridge_id` sideband.
- `NUM_MASTERS` / `NUM_SLAVES` — port counts.
- Localparam constants derived from the topology.
- Typedefs for the internal AXI4 structs used by the crossbar and adapters, such as `axi4_aw_t`, `axi4_w_<width>_t`, `axi4_b_t`, `axi4_ar_t`, `axi4_r_<width>_t`.

The exact set of typedefs depends on the unique data widths present in the bridge. For example, `bridge_mix_e` uses 32-bit and 64-bit paths, so its package defines `axi4_w_32b_t`, `axi4_r_32b_t`, `axi4_w_64b_t`, and `axi4_r_64b_t`.

17.2 Excerpt from a generated package

```
package bridge_mix_e_pkg;

    localparam int NUM_MASTERS = 3;
    localparam int NUM_SLAVES  = 4;  // includes subtractive catch-all
    localparam int BRIDGE_ID_WIDTH = 2;

    typedef struct packed {
        logic [3:0]  id;
        logic [31:0] addr;
        logic [7:0]  len;
        logic [2:0]  size;
        logic [1:0]  burst;
        logic        lock;
        logic [3:0]  cache;
        logic [2:0]  prot;
        logic [3:0]  qos;
        logic [3:0]  region;
        logic        user;
    } axi4_aw_t;

    // ... additional structs for W, B, AR, R at each used width ...

endpackage : bridge_mix_e_pkg
```

17.3 Why it matters

Because the top module imports this package, any change to widths or counts changes the package and therefore every generated file in the same variant. This is one reason partial regeneration is unsafe.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

18 Top Module

The top module (`<bridge_name>.sv`) is the integration shell. It instantiates the subtractive catch-all, every master adapter, the crossbar, and every slave adapter.

18.1 Module signature

The top module takes **no parameters**. All widths are fixed at generation time and come from the package:

```
module bridge_mix_e
    import bridge_mix_e_pkg::*;
(
    input  logic aclk,
    input  logic aresetn,

    // Master 0: cpu_axi3 (rw)
    input  logic [3:0] cpu_axi3_awid,
    input  logic [31:0] cpu_axi3_awaddr,
    // ... all AXI3/AXI4/APB/Wishbone channels per port protocol ...

    // Slave 0: ddr_axi3
    output logic [5:0] ddr_axi3_awid,
    // ...

    // Unmapped-address status (subtractive slave)
    output logic        unmapped_irq,
    output logic [31:0] unmapped_addr,
    output logic [7:0]  unmapped_count,
    input  logic        unmapped_clear
);
```

The exact port list is derived from the port protocol, channel subset, and widths.

18.2 Internal structure

The top module contains:

1. **`axi4_subtractive_slave`** — always present, terminates unmapped addresses with DECERR and exposes the `unmapped_*` status pins.
2. **Master adapter instances** — one per master, named `<name>_adapter`.
3. **Crossbar instance** — `<bridge_name>_xbar`, wired to all adapter outputs.
4. **Slave adapter instances** — one per external slave plus one `subtractive_adapter` for the catch-all.

18.3 Excerpt: subtractive catch-all instantiation

```
axi4_subtractive_slave #(
    .AXI_ID_WIDTH   (6),
    .AXI_ADDR_WIDTH (32),
    .AXI_DATA_WIDTH (64),
    .AXI_USER_WIDTH (1),
    .UNIT_ID        (8'd3),
    .AGENT_ID       (16'h5B00)
) u_subtractive (
    .aclk          (aclk),
    .aresetn       (aresetn),
    // ... AXI channels ...
    .o_hit_irq     (unmapped_irq),
    .o_hit_addr    (subtractive_hit_addr),
    .o_hit_count   (unmapped_count),
    .i_hit_clear   (subtractive_hit_clear)
);
```

18.4 Excerpt: adapter wiring pattern

```
cpu_axi3_adapter u_cpu_axi3_adapter (
    .aclk(aclk),
    .aresetn(aresetn),
    .cpu_axi3_awid(cpu_axi3_awid),
    // ... external AXI3 pins ...
    .slave_select_aw(cpu_axi3_slave_select_aw),
    .bridge_id_aw(cpu_axi3_bridge_id_aw),
    .slave_select_ar(cpu_axi3_slave_select_ar),
    .bridge_id_ar(cpu_axi3_bridge_id_ar),
    // ... width-specific internal paths ...
);
```

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

19 Adapters and Crossbar

19.1 Master adapter (<name>_adapter.sv)

The master adapter sits between an external master and the internal crossbar. Its responsibilities are:

- Expose the external port with the configured prefix and protocol.
- Insert timing-isolation wrappers when requested.
- Insert protocol converters for non-AXI4 master protocols (`apb`, `apb5`, `axil`, `axil5`, `wb4`, `axi3`, `axi5`).
- Generate per-width internal paths and instantiate width converters only where master and slave widths differ.
- Decode addresses and emit `slave_select_aw/ar` plus `bridge_id_aw/ar`.

19.1.1 Per-width paths

For each unique slave width the master connects to, the adapter emits a separate internal AXI4 path. If the master width matches the slave width, the path is direct; otherwise it uses a converter. Example from `bridge_mix_e`:

```
// cpu_axi3 Adapter outputs
logic [NUM_SLAVES-1:0] cpu_axi3_slave_select_aw;
logic [BRIDGE_ID_WIDTH-1:0] cpu_axi3_bridge_id_aw;
// 32b path
axi4_aw_t     cpu_axi3_32b_aw;
axi4_w_32b_t  cpu_axi3_32b_w;
// 64b path
axi4_aw_t     cpu_axi3_64b_aw;
axi4_w_64b_t  cpu_axi3_64b_w;
```

19.2 Slave adapter (<name>_adapter.sv)

The slave adapter sits between the crossbar and an external slave. Its responsibilities are:

- Convert the internal AXI4 face to the external protocol.
- Carry the `bridge_id` response tracker: `bridge_cam` or an in-order FIFO.
- Handle CDC if `cdc=true`.
- Expose APB, AXI-Lite, AXI3, AXI5, or Wishbone pins as configured.

19.2.1 Response routing

Each slave adapter stores the issuing master's `bridge_id` on the address handshake and hands it back with the response. For an AXI4/AXI5 slave in a bridge with more than one master (or any slave with `enable_ooo`) the store is a pair of `bridge_cam` instances keyed by the returned BID/RID, so the slave may complete different IDs out of order. Single-master bridges, protocol-shim slaves and the subtractive slave use an in-order FIFO whose head names the master. With more than one master, slave-side AXI IDs are `{BRIDGE_ID, id}`, `clog2(NUM_MASTERS)` bits wider than the master's.

19.3 Crossbar (<bridge_name>_xbar.sv)

The crossbar module:

- Accepts per-master decode outputs (`slave_select_*`, `bridge_id_*`).
- Builds AW/AR request matrices.
- Instantiates per-slave round-robin arbiters with grant locked to the address handshake.
- Routes W beats via a per-slave W-owner FIFO.
- Muxes responses back to masters using the `bridge_id` sideband.
- Optionally inserts a registered stage when `xbar_pipeline=true`.

The crossbar is protocol-agnostic internally; all protocol conversion happens in the adapters.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

20 Filelist

The filelist (`<bridge_name>.f`) declares the complete compile closure for one variant. It is generated in `bridge_generator.py` by `_emit_bridge_variant()`.

20.1 Path convention

All paths inside the filelist are anchored to `$REPO_ROOT`. This allows the filelist to be included from another filelist regardless of the including file's base directory.

20.2 Example filelist head

```
# Include directories
+incdir+$REPO_ROOT/rtl/amba/includes

# Reset macro header (`ALWAYS_FF_RST / `RST_ASSERTED)
-f $REPO_ROOT/rtl/amba/filelists/reset_defs.f

# Monitor packages (must precede any module that references them)
-f $REPO_ROOT/rtl/amba/filelists/monitor_pkgs.f

# Subtractive catch-all slave (unmapped-address terminator)
-f $REPO_ROOT/rtl/amba/filelists/axi4_subtractive_slave.f

# Bridge RTL files (generated)
$REPO_ROOT/projects/components/fabric-gen-ip/bridge/rtl/generated/
bridge_mix_e/bridge_mix_e_pkg.sv
$REPO_ROOT/projects/components/fabric-gen-ip/bridge/rtl/generated/
bridge_mix_e/bridge_mix_e_xbar.sv
$REPO_ROOT/projects/components/fabric-gen-ip/bridge/rtl/generated/
bridge_mix_e/bridge_mix_e.sv
$REPO_ROOT/projects/components/fabric-gen-ip/bridge/rtl/generated/
bridge_mix_e/apb_periph_adapter.sv
...

# AXI4 Wrapper modules (timing isolation)
-f $REPO_ROOT/rtl/amba/filelists/axi4_slave_wr.f
-f $REPO_ROOT/rtl/amba/filelists/axi4_slave_rd.f
-f $REPO_ROOT/rtl/amba/filelists/axi4_master_wr.f
-f $REPO_ROOT/rtl/amba/filelists/axi4_master_rd.f

# Width converters
-f
$REPO_ROOT/projects/components/utility-ip/converters/rtl/filelists/
axi_data_upsize.f
-f
$REPO_ROOT/projects/components/utility-ip/converters/rtl/filelists/
axi_data_dnsize.f
-f
$REPO_ROOT/projects/components/utility-ip/converters/rtl/filelists/
axi4_dwidth_converter_rd.f
-f
$REPO_ROOT/projects/components/utility-ip/converters/rtl/filelists/
axi4_dwidth_converter_wr.f
```

20.3 Filelist is generated, not hand-maintained

Because the converter and wrapper selections depend on the port protocols, the filelist is regenerated every time the bridge is regenerated. Do not hand-edit a generated filelist; instead, change the generator's selection logic and regenerate.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

21 Generated Tests

When `--generate-tests` is passed, the generator emits a cocotb testbench class and a pytest runner for each variant.

21.1 TB class

The TB class is written to `dv/tbclasses/<tb_class_module>.py`. Its name is derived from the bridge name:

```
bridge_2x2_rw  → Bridge2x2RwTB
bridge_mix_e   → BridgeMixETB
```

The class inherits from `TBBase` and implements:

- `setup_clocks_and_reset()` — drives `aclk` and `aresetn`.
- `assert_reset()` — drives `self.dut.aresetn.value = 0`.
- `deassert_reset()` — releases reset.

It constructs `GAXIMaster` / `GAXISlave` BFMs per port using the AXI4 field configs derived from the generated widths.

21.2 Test runner

The test runner (`dv/tests/test_<bridge>.py`) contains `cocotb.test` functions and pytest wrappers. Each wrapper names its `testcase=` explicitly. Example shape:

```
@cocotb.test()
async def basic_read_write(dut):
    tb = Bridge2x2RwTB(dut)
    await tb.run_basic_read_write()

@pytest.mark.parametrize("seed", [0, 1, 2])
def test_bridge_2x2_rw_basic(seed):
    run_test(
        module="test_bridge_2x2_rw",
        testcase="basic_read_write",
        extra_env={"SEED": str(seed)}
    )
```

21.3 Monitor stress test

For "mon" variants, `generate_monitor_tests()` emits:

- A TB class with the full monbus/cfg surface.
- `test_<bridge>_monitor.py` driving backpressure and error injection phases scaled by `TEST_LEVEL`.

21.4 Regeneration discipline for tests

Tests are generated artifacts. After any change to the test templates or to widths that affect BFM construction, regenerate tests with `--generate-tests` from a clean state. Hand-written tests for generated DUTs should live in separate files (for example, `test_bridge_2x2_rw_tracking.py`) and not be overwritten by the generator.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

22 Full Regeneration Rule

Generated RTL must be treated like compiler output: when the compiler changes, rebuild everything.

22.1 Why partial regeneration is dangerous

- The package file defines widths used by every other generated file.
- Converter selection logic affects adapter port lists and the filelist.
- The `bridge_id` width is global; a change changes the crossbar and every adapter.
- The filelist pulls in external IP closures; an interface change there must be reflected in every generated bridge.

22.2 Required workflow

```
cd projects/components/fabric-gen-ip/bridge/bin
make clean
make all
```

If tests are generated:

```
python bridge_generator.py --bulk bridge_batch.csv --generate-tests
cd ../dv/tests
make clean-all && make run-all-func
```

22.3 Code changes that trigger full regeneration

- `bin/bridge_generator.py`
- Any file under `bin/bridge_pkg/`
- Any Jinja template under `bin/bridge_pkg/jinja_templates/`
- Any shared converter/wrapper IP whose interface changed

22.4 Files that are never regenerated

- `rtl/bridge_cam.sv` — hand-written CAM, instantiated by the slave adapters of multi-master AXI4/AXI5 slaves.
- Anything outside `rtl/generated/` and `rtl/filelists/`.

22.5 Version-mismatch symptoms

- Tests that previously passed now fail.
- Simulation errors about missing signals or mismatched widths.
- Address decode routing to wrong slaves.
- Verilator warnings about undeclared identifiers in the package.

If any of these appear after a generator change, the first recovery step is `make clean && make all`.

---

23 Generated Examples

The checked-in `rtl/generated/` directory contains a variety of fixtures that serve as regression anchors and integration examples.

23.1 Representative fixtures

Representative fixtures

Fixture	Masters	Slaves	Notable features
`bridge_2x2_rw`	2	2	Plain AXI4, 32-bit addresses, per-port widths.
`bridge_2x2_axi5`	2	2	Native AXI5 fabric with MTE/chunking.
`bridge_1x2_rw_apb5`	1	2	APB5 slave, single master.
`bridge_2x2_axi3_masters`	2	2	AXI3 masters on AXI4 fabric.
`bridge_mix_e`	3	3	AXI3 + AXI4 + APB masters → AXI3 + AXI4 + APB slaves.
`bridge_1x2_rd_regblock_mon`	1	2	`use_cfg_regblock=true`, `"mon"` variant.
`bridge_2x2_rw_qos_pipe`	2	2	`arbitration="qos"`, `xbar_pipeline=true`.

23.2 How to verify a regeneration

After `make all`, every fixture in `rtl/generated/` should be byte-identical to the checked-in version **if** the generator and inputs have not changed. Because the generated files carry a generation-date-free header, a clean regeneration should not introduce spurious diffs.

To compare:

```
cd projects/components/fabric-gen-ip/bridge
# Regenerate everything
make -C bin clean && make -C bin all
# Check for unexpected diffs
git diff -- rtl/generated/ | head -n 200
```

(Note: the user instruction forbids running git commands; the command above is for illustration only and is the responsibility of the agent that owns the shared tree.)

If the generator code changed intentionally, the diffs are expected and must be reviewed.

23.3 Fixture as documentation

The `bridge_mix_e` fixture is the canonical mixed-protocol example. Its TOML and generated top module are referenced throughout this specification. To understand how a new protocol or width combination is emitted, generate a minimal fixture and compare it to the closest existing one.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

24 DV Suite

Bridge uses a cocotb/pytest DV suite in `dv/tests/`. Test classes live in `dv/tbclasses/`; runners live in `dv/tests/`.

24.1 Test structure

```
dv/
├── tbclasses/
│   ├── bridge2x2_rw_tb.py          # per-config TB class
│   ├── bridge_mix_e_tb.py
│   └── ...
└── tests/
    ├── conftest.py
    ├── test_bridge_2x2_rw.py       # generated
    ├── test_bridge_mix_e.py        # generated
    ├── test_bridge_2x2_rw_tracking.py   # hand-written
    ├── test_bridge_1x2_rw_axi5a_atomics.py
    └── ...
```

24.2 Running tests

Use the area Makefile, not bare pytest:

```
cd projects/components/fabric-gen-ip/bridge/dv/tests

# Specific test, gate level
make run-bridge_2x2_rw-gate

# All functional tests
make clean-all && make run-all-func

# Full regression
make clean-all && make run-all-full
```

Bridge tests run serially because each test consumes approximately 1 GB of memory.

24.3 Test levels

Each generated wrapper exports `REG_LEVEL` cells:

Test levels

Level	Cells	Description
GATE	72	Smoke / gate-level depth
FUNC	144	Functional depth
FULL	216	Stress depth

The TB reads the depth profile from `dv/tbclasses/bridge_levels.py`.

24.4 Verification coverage

The DV suite covers:

- Basic read/write connectivity per master-slave pair.
- Boundary probe (address decode miss and subtractive catch-all).
- Arbitration fairness and multi-master contention.
- Burst transfers and packet atomicity.
- Width conversion paths.
- APB/AXI-Lite/AXI3/Wishbone slave conversions.
- AXI5 sideband propagation and atomics.
- Monitor stress (`*_mon_monitor.py`) and cfg regblock programming.

24.5 When generator changes affect DV

After any generator change that affects the generated port list, width, or filelist, regenerate the tests and run `make clean-all && make run-all-func` before declaring the change safe.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

25 Lint and Synthesis Checks

Every generated bridge is expected to pass Verilator lint and out-of-context synthesis.

25.1 Verilator lint

The `rtl/Makefile` runs lint:

```
cd projects/components/fabric-gen-ip/bridge/rtl
make verilator
```

This lints every generated fixture using its filelist. The generator aims for lint-clean output; any new warning class must be either fixed in the generator or documented with a scoped waiver.

25.2 Out-of-context synthesis

The FPGA flow in `projects/components/fabric-gen-ip/bridge/fpga/` supports out-of-context synthesis:

```
cd projects/components/fabric-gen-ip/bridge/fpga
make synth BRIDGE=bridge_2x2_rw PART=xc7a100tcsg324-1 CLK_NS=10
```

Resource and timing tables in the HAS/MAS are produced by this flow.

25.3 When to run checks

Run lint and at least one synthesis check after:

- Adding a new protocol or option.
- Changing the adapter or crossbar emission templates.
- Updating converter/wrapper dependencies.

A generator change that passes simulation but fails synthesis is not done.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

26 Adding a Protocol

Adding a new protocol to Bridge means adding a new value to the `protocol` enum, validating it, selecting the right converter/wrapper in the adapters, and updating the filelist logic.

26.1 Step 1: Add the protocol to the validator

In `bridge_pkg/config_validator.py`:

1. Add the protocol string to `valid_protocols` in `validate_protocol()`.
2. Add a new `validate_<proto>_constraints()` function if the protocol has special rules (for example, allowed widths, channel restrictions, ID width rules).
3. Call the new validator from `validate_config()`.

26.2 Step 2: Add converter/wrapper selection

In `bridge_pkg/generators/adapter_generator.py`:

- For a master protocol, decide which front-end converter module to instantiate.
- Add the protocol to the width-converter path selection.
- Emit the correct external port signals.

In `bridge_pkg/generators/slave_adapter_generator.py`:

- For a slave protocol, decide which completer shim to instantiate.
- Wire the internal AXI4 face to the shim.

26.3 Step 3: Update the filelist

In `bridge_generator.py` `_emit_bridge_variant()`:

- Add `if any(m.protocol.lower() == '<proto>' for m in config.masters):` block to pull in the master-side converter closure.
- Add the corresponding slave-side block.

Always `-f` the converter's own closure filelist; do not hand-list individual `.sv` files.

26.4 Step 4: Add a representative fixture

Add a TOML/CSV pair to `bin/test_configs/` and a line to `bridge_batch.csv` so the new protocol is exercised in regression.

26.5 Step 5: Run the full regeneration and test

```
cd projects/components/fabric-gen-ip/bridge/bin
make clean && make all
python bridge_generator.py --bulk bridge_batch.csv --generate-tests
cd ../dv/tests
make clean-all && make run-<new_fixture>-func
```

26.6 Example: how APB was added

APB master support uses `apb4_to_axi4` / `apb5_to_axi4` front-end converters. APB slave support uses `axi4_to_apb4_shim` / `axi4_to_apb5_shim`. The validator enforces `channels="rw"`, `data_width` in `{8,16,32}`, and `id_width=0`. The filelist pulls in the converter closures conditionally.

---

27 Adding a Bridge-Level Option

Bridge-level options live in the `[bridge]` TOML table and become fields of `BridgeConfig`.

27.1 Step 1: Add the dataclass field

In `bridge_pkg/config.py`, add a field to `BridgeConfig` with a sensible default:

```
@dataclass
class BridgeConfig:
    ...
    my_option: bool = False
```

27.2 Step 2: Add the TOML loader mapping

In `bridge_pkg/config_loader.py`, read the new key from the TOML tree and assign it to the `BridgeConfig` instance.

27.3 Step 3: Validate if necessary

If the option has legal-value restrictions or conflicts with other options, add a validator in `bridge_pkg/config_validator.py` and call it from `validate_config()`.

27.4 Step 4: Thread it into the emitter

In `bridge_generator.py` `_emit_bridge_variant()`, pass the option to `BridgeModuleGenerator`:

```
gen = BridgeModuleGenerator(
    ...
    my_option=getattr(config, 'my_option', False),
)
```

27.5 Step 5: Use the option in `BridgeModuleGenerator`

In `bridge_pkg/components/bridge_module_generator.py`:

1. Accept the option in `__init__`.
2. Store it as an instance attribute.
3. Use it to conditionally emit logic, instantiate modules, or modify the port list.

27.6 Step 6: Document and test

- Add the key to this GAS chapter 2.
- Add a fixture to `bridge_batch.csv` that exercises the option.
- Regenerate and run the DV suite.

27.7 Example: xbar_pipeline

xbar_pipeline is a bridge-level option that adds a registered skid stage inside the crossbar. It is loaded from TOML, stored in BridgeConfig, passed to BridgeModuleGenerator, and used by CrossbarGenerator to emit extra pipeline registers.

28 Adding a Port-Level Option

Port-level options are fields of `PortSpec`. They affect one port at a time.

28.1 Step 1: Add the PortSpec field

In `bridge_pkg/config.py`, add a field to `PortSpec` with a default:

```
@dataclass
class PortSpec:
    ...
    my_port_option: bool = False
```

28.2 Step 2: Load it from TOML

In `bridge_pkg/config_loader.py`, copy the TOML key into the `PortSpec` constructor for both masters and slaves.

28.3 Step 3: Validate it

If the option has constraints, add a validator in `bridge_pkg/config_validator.py`. For example, `cdc` is validated by `validate_cdc_constraints()` to ensure it is only used on AXI4 slave ports.

28.4 Step 4: Thread it into `MasterConfig` / `SlaveInfo`

In `bridge_generator.py` `_emit_bridge_variant()`, pass the option through when building `MasterConfig` or `SlaveInfo`:

```
master_config = MasterConfig(
    ...
    my_port_option=getattr(master_spec, 'my_port_option', False),
)
```

28.5 Step 5: Use it in the adapter generator

In `bridge_pkg/generators/adapter_generator.py` or `slave_adapter_generator.py`, read the option from the config struct and emit conditional logic.

28.6 Step 6: Update the filelist if needed

If the option requires additional IP, add conditional filelist lines in `bridge_generator.py`.

28.7 Example: cdc

The `cdc` port option is a `PortSpec` field. It is validated as AXI4-slave-only, passed through `SlaveInfo`, and used by `SlaveAdapterGenerator` to insert `axi4_cdc_{wr,rd}`. The filelist conditionally pulls in `axi4_cdc_wr.f` and `axi4_cdc_rd.f`.

29 Testing an Extension

Any generator extension must be proven by regeneration, lint, and DV.

29.1 Minimum test plan

1. **Single-bridge generation**

   ```
   python bridge_generator.py --ports test_configs/my_extension.toml
   ```

   Verify the output directory contains the expected files and that the new logic appears in the generated `.sv`.

2. **Validation failure paths**

   Create deliberately invalid TOML files and confirm the generator raises `ValidationError` with a clear message.

3. **Bulk regeneration**

   ```
   make clean && make all
   ```

   Confirm no fixture regresses.

4. **Lint**

   ```
   cd ../rtl && make verilator
   ```

5. **DV smoke**

   ```
   cd ../dv/tests
   make clean-all && make run-<fixture>-func
   ```

6. **Synthesis sanity**

   For area-sensitive options, run out-of-context synthesis on the smallest fixture that exercises the option.

29.2 Regression fixtures

Add at least one fixture to `bridge_batch.csv` that exercises the new protocol or option. If the option is off by default, also add a fixture with the option enabled.

29.3 Documentation

Update this GAS and `CLAUDE.md` if the extension changes the way users configure or extend the generator. Keep the validator rule list, the configuration tables, and the extension-guide steps in sync.